Micro-SIEM – Image Prompts

Every image needed for the project report (Micro-SIEM.docx) and presentation (Micro-SIEM.pptx)

How to use this file

1. **Part A – 8 diagrams.** Each has two ways to make it:
   - **AI PROMPT** copy it into an AI image/diagram generator (ChatGPT, Gemini, Copilot, Eraser.io, draw.io → Arrange → Insert → AI). Always paste the *style line* below together with the prompt.
   - **CODE** the exact diagram as code – most reliable, labels are always correct. **Mermaid** → paste at **mermaid.live** → Actions → PNG. **PlantUML** → paste at **planttext.com** (or plantuml.com/plantuml) → PNG.
2. **Part B – 13 screenshots.** These must be **real screenshots** of your running system (start it with `docker-compose up -d`). Do not use AI-generated fake screenshots in a project report – examiners can tell, and it would misrepresent your work.
3. In Word, click the dashed placeholder box → delete its text → Insert → Pictures, and size the image to the box. If a diagram is taller than the box, drag the box taller and re-check the index page numbers.
4. Check AI output carefully: AI image tools often misspell labels or invent arrows. If anything is wrong, use the code version.

Style line – add to the end of every AI prompt

> Style: clean, professional, flat vector diagram on a pure white background; dark navy (#0B1F33) text and outlines, teal (#1FA2A6) for system components, red (#E63946) for attacker/threat elements; sans-serif font; every label spelled exactly as given; no decorative icons, no 3D, no shadows, no extra text or watermarks; high resolution PNG.

Checklist

Figure	Image	Type	Where it goes
Figure 3.1	System Architecture of Micro-SIEM	DIAGRAM	Report §3.1 (page 7) · optional replacement for Slide 8
Figure 4.1	Iterative and Incremental Model	DIAGRAM	Report §4.2 (page 13)
Figure 5.1	Use Case Diagram	DIAGRAM	Report §5.1 (page 16) · matches Slide 11
Figure 5.2	Class Diagram	DIAGRAM	Report §5.2 (page 18)
Figure 5.3	Activity Diagram – Demo Login and Logging	DIAGRAM	Report §5.3 (a) (page 19)

Figure 5.4	Activity Diagram – Detection Cycle	DIAGRAM	Report §5.3 (b) (page 20)
Figure 5.5	Activity Diagram – Dashboard Login and Role Check	DIAGRAM	Report §5.3 (c) (page 20)
Figure 5.6	Deployment Diagram	DIAGRAM	Report §5.4 (page 21)
Figure 6.1	Demo Portal – Home Page	SCREENSHOT	Report p.28
Figure 6.2	Demo Login – Access Denied	SCREENSHOT	Report p.29 · Slide 13
Figure 6.3	Search Page with XSS Payload	SCREENSHOT	Report p.29 · Slide 13
Figure 6.4	Dashboard Sign-In Page	SCREENSHOT	Report p.30
Figure 6.5	Admin Dashboard – Overview	SCREENSHOT	Report p.30 · Slide 14
Figure 6.6	Admin Dashboard – All Logs View	SCREENSHOT	Report p.31
Figure 6.7	Admin Dashboard – Threats & Action	SCREENSHOT	Report p.31 · Slide 15
Figure 6.8	Coming Soon Pop-up	SCREENSHOT	Report p.32 · Slide 15
Figure 6.9	Viewer Dashboard (Read-Only)	SCREENSHOT	Report p.32
Figure 6.10	OpenSearch Dashboards – Raw Events	SCREENSHOT	Report p.33
Figure 6.11	Running Containers	SCREENSHOT	Report p.33
Figure 6.12	FastAPI API Documentation	SCREENSHOT	Report p.34
Figure 7.1	Alerts Raised During Attack Simulation	SCREENSHOT	Report p.37

Page numbers refer to the page number printed at the bottom of the report pages. Slide 8 (architecture) is already drawn in the presentation; Figure 3.1 can optionally replace it.

Part A – Diagrams

Figure 3.1 – System Architecture of Micro-SIEM

Used in: Report §3.1 (page 7) · optional replacement for Slide 8 · **Size:** Landscape 3:2 (e.g. 1800 × 1200 px)

AI PROMPT **copy this + the style line**

> Create a system architecture block diagram titled "Micro-SIEM – System Architecture", flowing left to right.
> LEFT column, grouped in a box labelled "Log Sources": three rectangles – "Web Login Log (/var/log/webapp/login.log)", "Nginx Access Log (access_json.log)", "SSH auth.log (/var/log/auth.log)".
> Arrows from all three into a teal rounded box "Vector – parse + tag (log_type, source_ip)".
> Arrow from Vector into a navy box "OpenSearch" that contains three database cylinders: "logs-index", "alerts-index", "blocked-index".
> Arrow labelled "query last 2 min" from logs-index to a red box "FastAPI – Detection Engine + Auth API (:8000)"; arrow labelled "write alerts" from FastAPI back to alerts-index; a two-way arrow between FastAPI and blocked-index.
> From FastAPI, two arrows to light-grey boxes "Admin Dashboard (:8090)" and "Viewer Dashboard (:8090)".
> BOTTOM row: a red-outlined box "Attacker Browser" with an arrow to "Nginx Demo Portal (:8080)"; arrow labelled "/api/login" from the portal to FastAPI; dashed arrow labelled "writes" from the portal to "Nginx Access Log"; dashed arrow labelled "writes" from FastAPI to "Web Login Log".

CODE · Mermaid **paste at mermaid.live**

```
flowchart LR
  subgraph SRC[Log Sources]
    A1[Web Login Log<br/>/var/log/webapp/login.log]
    A2[Nginx Access Log<br/>access_json.log]
    A3[SSH auth.log<br/>/var/log/auth.log]
  end
  V[Vector<br/>parse + tag log_type, source_ip]
  subgraph OS[OpenSearch]
    I1[(logs-index)]
    I2[(alerts-index)]
    I3[(blocked-index)]
  end
  API[FastAPI<br/>Detection Engine + Auth API<br/>:8000]
  AD[Admin Dashboard :8090]
  VD[Viewer Dashboard :8090]
  ATK([Attacker Browser])
  NG[Nginx Demo Portal :8080]
  A1 & A2 & A3 --> V --> I1
  I1 -->|query last 2 min| API
  API -->|write alerts| I2
  API <--> I3
  API --> AD
  API --> VD
  ATK --> NG
  NG -->|/api/login| API
  NG -.->|writes| A2
  API -.->|writes| A1
```

Figure 4.1 – Iterative and Incremental Model

Used in: Report §4.2 (page 13) · **Size:** Landscape 2:1 (e.g. 1800 × 900 px)

AI PROMPT **copy this + the style line**

> Create a software-engineering diagram titled "Iterative and Incremental Model".
> Show five phases as rounded boxes arranged in a circular cycle with arrows: "Requirements" → "Design" → "Implementation" → "Testing" → "Review" → back to "Requirements" (label this return arrow "next increment").
> To the right of the cycle, a vertical stack of seven small labelled boxes: "Increment 1: Docker + OpenSearch + Vector ingestion", "Increment 2: Brute-force detection + alert history", "Increment 3: Nginx logs, XSS + correlation rules", "Increment 4: Admin & viewer dashboards", "Increment 5: Session auth + role-based access", "Increment 6: Demo portal + response actions", "Increment 7: Testing, fixes, documentation".
> An arrow from "Review" to a final teal box "Final System: Micro-SIEM".

CODE · Mermaid **paste at mermaid.live**

```
flowchart LR
  R[Requirements] --> D[Design] --> I[Implementation] --> T[Testing] --> E[Review]
  E -->|next increment| R
  E -->|after increment 7| F([Final System: Micro-SIEM])
  subgraph INC[Increments]
    direction TB
    I1[1: Docker + OpenSearch + Vector ingestion]
    I2[2: Brute-force detection + alert history]
    I3[3: Nginx logs, XSS + correlation rules]
    I4[4: Admin and viewer dashboards]
    I5[5: Session auth + role-based access]
    I6[6: Demo portal + response actions]
    I7[7: Testing, fixes, documentation]
    I1 --> I2 --> I3 --> I4 --> I5 --> I6 --> I7
  end
```

Figure 5.1 – Use Case Diagram

Used in: Report §5.1 (page 16) · matches Slide 11 · **Size:** Landscape 5:4 (e.g. 1500 × 1200 px)

AI PROMPT **copy this + the style line**

Create a UML use case diagram. A large system-boundary rectangle labelled "Micro-SIEM".
Stick-figure actors OUTSIDE the boundary: "Admin" and "Viewer" on the left; "Demo User / Attacker" and "System (Vector + Detection Engine)" on the right.
Ovals (use cases) INSIDE the boundary: "Login to Dashboard", "View Dashboard Statistics", "View Logs", "View Alerts / Threats", "View Blocked List", "Block IP / Account «future»", "Logout", "Login to Demo Portal", "Search on Demo Portal", "Collect Logs", "Detect Attacks", "Generate Alert".
Lines: Admin – Login to Dashboard, View Dashboard Statistics, View Logs, View Alerts / Threats, View Blocked List, Block IP / Account «future», Logout. Viewer – Login to Dashboard, View Dashboard Statistics, Logout. Demo User / Attacker – Login to Demo Portal, Search on Demo Portal. System – Collect Logs, Detect Attacks, Generate Alert.
Dashed arrows labelled «include»: View Alerts / Threats → Detect Attacks; Detect Attacks → Generate Alert.

CODE · PlantUML **paste at planttext.com**

```
@startuml
left to right direction
skinparam packageStyle rectangle
actor Admin
actor Viewer
actor "Demo User /\nAttacker" as Attacker
actor "System\n(Vector + Engine)" as System
rectangle "Micro-SIEM" {
  usecase "Login to Dashboard" as UC1
  usecase "View Dashboard Statistics" as UC2
  usecase "View Logs" as UC3
  usecase "View Alerts / Threats" as UC4
  usecase "View Blocked List" as UC5
  usecase "Block IP / Account\n<<future>>" as UC6
  usecase "Logout" as UC7
  usecase "Login to Demo Portal" as UC8
  usecase "Search on Demo Portal" as UC9
  usecase "Collect Logs" as UC10
  usecase "Detect Attacks" as UC11
  usecase "Generate Alert" as UC12
}
Admin --> UC1
Admin --> UC2
Admin --> UC3
Admin --> UC4
Admin --> UC5
Admin --> UC6
Admin --> UC7
Viewer --> UC1
Viewer --> UC2
Viewer --> UC7
Attacker --> UC8
Attacker --> UC9
System --> UC10
System --> UC11
System --> UC12
UC4 ..> UC11 : <<include>>
UC11 ..> UC12 : <<include>>
@enduml
```

Figure 5.2 – Class Diagram

Used in: Report §5.2 (page 18) · **Size:** Landscape 4:3 (e.g. 1600 × 1200 px)

AI PROMPT **copy this + the style line**

> Create a UML class diagram with eight classes, each a three-compartment box (name / attributes / methods):
> 1. "LoginRequest": username: str, password: str.
> 2. "DashboardLoginRequest": username: str, password: str.
> 3. "Session": token: str, role: str, expires: datetime | require_session(), require_admin().
> 4. "LogEvent": timestamp, log_type, source_ip, message, event, username, path, status.
> 5. "DetectionRule": alert_type, rule_name, match_field, match_value, mitre_id, mitre_tactic | _run_detection(), _run_correlation_rule(), _run_xss_detection().
> 6. "Alert": timestamp, alert_type, source_ip, attempt_count, severity, matched_rule, mitre_technique, mitre_id, mitre_tactic | _already_alerted_recently().
> 7. "BlockRecord": type, ip, username, blocked_at, blocked_by | block_ip(), unblock_ip(), block_account(), unblock_account().
> 8. "SIEMApi" (central): client: OpenSearch, sessions, demo_sessions | web_login(), dashboard_login(), get_logs(), get_alerts(), get_blocked().
> Relationships: SIEMApi --> LoginRequest and DashboardLoginRequest (dashed, "validates"); SIEMApi 1 → * Session ("manages"); SIEMApi → DetectionRule ("runs"); DetectionRule 1 → * LogEvent ("reads"); DetectionRule 1 → * Alert ("creates"); SIEMApi → BlockRecord ("manages").

CODE · Mermaid **paste at mermaid.live**

```
classDiagram
  class LoginRequest {
    +str username
    +str password
  }
  class DashboardLoginRequest {
    +str username
    +str password
  }
  class Session {
    +str token
    +str role
    +datetime expires
    +require_session()
    +require_admin()
  }
  class LogEvent {
    +datetime timestamp
    +str log_type
    +str source_ip
    +str message
    +str event
    +str username
    +str path
    +int status
  }
  class DetectionRule {
    +str alert_type
    +str rule_name
    +str match_field
    +str match_value
    +str mitre_id
    +str mitre_tactic
    +run_detection()
    +run_correlation_rule()
```

```
  +run_xss_detection()
}
class Alert {
  +datetime timestamp
  +str alert_type
  +str source_ip
  +int attempt_count
  +str severity
  +str matched_rule
  +str mitre_technique
  +str mitre_id
  +str mitre_tactic
  +already_alerted_recently()
}
class BlockRecord {
  +str type
  +str ip
  +str username
  +datetime blocked_at
  +str blocked_by
  +block_ip()
  +unblock_ip()
  +block_account()
  +unblock_account()
}
class SIEMApi {
  +OpenSearch client
  +dict sessions
  +dict demo_sessions
  +web_login()
  +dashboard_login()
  +get_logs()
  +get_alerts()
  +get_blocked()
}
SIEMApi ..> LoginRequest : validates
SIEMApi ..> DashboardLoginRequest : validates
SIEMApi "1" --> "*" Session : manages
SIEMApi --> DetectionRule : runs
DetectionRule "1" --> "*" LogEvent : reads
DetectionRule "1" --> "*" Alert : creates
SIEMApi --> BlockRecord : manages
```

Figure 5.3 – Activity Diagram – Demo Login and Logging

Used in: Report §5.3 (a) (page 19) · **Size:** Portrait or square (e.g. 1200 × 1400 px) – enlarge the box in Word if needed

AI PROMPT **copy this + the style line**

> Create a UML activity diagram titled "Demo Login and Logging", top to bottom. Use a filled black circle for Start and a bullseye for End, rounded rectangles for actions and diamonds for decisions.
> Start → "Enter username and password" → "Submit to /api/login" → decision "Account blocked?".
> If Yes → "Write login_blocked event" → "Show 'account blocked by administrator'" → End.
> If No → decision "Password correct?".
> If Yes → "Write login_success event" → "Create demo session token" → "Redirect to search page" → End.
> If No → "Write login_failed event" → "Show 'Access denied – invalid credentials'" → End.
> Add a separate side lane labelled "Vector" with one action "Ship new log line to OpenSearch (logs-index)", connected by dashed arrows from all three "Write … event" actions.

CODE · Mermaid **paste at mermaid.live**

```
flowchart TD
  S([Start]) --> A[Enter username and password]
  A --> B[Submit to /api/login]
  B --> C{Account blocked?}
  C -- Yes --> D[Write login_blocked event]
  D --> E[Show 'account blocked by administrator'] --> Z([End])
  C -- No --> F{Password correct?}
  F -- Yes --> G[Write login_success event]
  G --> H[Create demo session token] --> I[Redirect to search page] --> Z
  F -- No --> J[Write login_failed event]
  J --> K[Show 'Access denied - invalid credentials'] --> Z
  D & G & J -.-> V[Vector ships log line to OpenSearch logs-index]
```

Figure 5.4 – Activity Diagram – Detection Cycle

Used in: Report §5.3 (b) (page 20) · **Size:** Portrait or square (e.g. 1200 × 1500 px) – enlarge the box in Word if needed

AI PROMPT **copy this + the style line**

Create a UML activity diagram titled "Detection Cycle", top to bottom, with loops.
Start ("Dashboard calls /alerts every 3 seconds") → "Take next detection rule" → "Query events of the last 2 minutes" → "Group events by source IP" → decision "Rule matched for this IP? (threshold / sequence / signature)".
If No → decision "More IPs?".
If Yes → decision "XSS rule and already alerted in last 5 minutes?": Yes → "More IPs?"; No → "Write alert to alerts-index" → "More IPs?".
"More IPs?": Yes → loop back to the "Rule matched…" decision; No → decision "More rules?".
"More rules?": Yes → loop back to "Take next detection rule"; No → "Return latest 50 alerts" → "Dashboard updates charts and tables" → End.

CODE · Mermaid **paste at mermaid.live**

```
flowchart TD
  S([Dashboard calls /alerts every 3 s]) --> R[Take next detection rule]
  R --> Q[Query events of last 2 minutes]
  Q --> G[Group events by source IP]
  G --> M{Rule matched for this IP?<br/>threshold / sequence / signature}
  M -- No --> N{More IPs?}
  M -- Yes --> X{XSS rule and alerted<br/>in last 5 min?}
  X -- Yes --> N
  X -- No --> W[Write alert to alerts-index]
  W --> N
  N -- Yes --> M
  N -- No --> MR{More rules?}
  MR -- Yes --> R
  MR -- No --> L[Return latest 50 alerts]
  L --> U[Dashboard updates charts and tables] --> E([End])
```

Figure 5.5 – Activity Diagram – Dashboard Login and Role Check

Used in: Report §5.3 (c) (page 20) · **Size:** Portrait or square (e.g. 1200 × 1400 px) – enlarge the box in Word if needed

AI PROMPT **copy this + the style line**

Create a UML activity diagram titled "Dashboard Login and Role Check", top to bottom.
Start → "Enter username and password" → decision "Valid credentials?".
No → "Show 'Invalid username or password'" → End.
Yes → "Create session token (valid 8 hours)" → decision "Role = admin?": Yes → "Open admin.html"; No → "Open viewer.html".
Both merge into "Every API call sends Bearer token" → decision "Token valid and not expired?".
No → "HTTP 401 – redirect to login page" → End.
Yes → decision "Admin-only action and role = viewer?": Yes → "HTTP 403 Forbidden" → End; No → "Return requested data" → End.

CODE · Mermaid **paste at mermaid.live**

```
flowchart TD
  S([Start]) --> A[Enter username and password]
  A --> B{Valid credentials?}
  B -- No --> C[Show 'Invalid username or password'] --> E([End])
  B -- Yes --> D[Create session token - valid 8 hours]
  D --> F{Role = admin?}
  F -- Yes --> G[Open admin.html]
  F -- No --> H[Open viewer.html]
  G & H --> I[Every API call sends Bearer token]
  I --> J{Token valid and not expired?}
  J -- No --> K[HTTP 401 - redirect to login] --> E
  J -- Yes --> L{Admin-only action<br/>and role = viewer?}
  L -- Yes --> M[HTTP 403 Forbidden] --> E
  L -- No --> N[Return requested data] --> E
```

Figure 5.6 – Deployment Diagram

Used in: Report §5.4 (page 21) · **Size:** Landscape 3:2 (e.g. 1800 × 1200 px)

AI PROMPT **copy this + the style line**

> Create a UML deployment diagram. One large 3D node box labelled "Linux Host (Docker Engine)".
> Inside it, a nested node "siem-net (Docker bridge network)" containing five container nodes: "opensearch :9200" (drawn as a database), "opensearch-dashboards :5601", "vector", "nginx-loginpage :8080", "react-console :8090".
> Also inside the host but OUTSIDE the bridge network: node "fastapi-app :8000 (host network, NET_ADMIN)".
> Inside the host: two storage cylinders "webapp-logs volume" and "nginx-logs volume", and a file icon "/var/log/auth.log".
> Arrows: fastapi-app → webapp-logs ("writes login.log"); nginx-loginpage → nginx-logs ("writes access_json.log"); webapp-logs, nginx-logs and /var/log/auth.log → vector; vector → opensearch ("bulk insert"); opensearch-dashboards → opensearch; fastapi-app → opensearch ("queries / alerts"); nginx-loginpage → fastapi-app ("/api/ proxy").
> Outside the host: actor "Admin / Viewer Browser" with arrows to react-console (":8090") and fastapi-app (":8000"); actor "Attacker Browser" with arrow to nginx-loginpage (":8080").

CODE · PlantUML **paste at planttext.com**

```
@startuml
node "Linux Host (Docker Engine)" {
  node "siem-net (bridge network)" {
    database "opensearch\n:9200" as OS
    node "opensearch-dashboards\n:5601" as OSD
    node "vector" as VEC
    node "nginx-loginpage\n:8080" as NG
    node "react-console\n:8090" as RC
  }
  node "fastapi-app :8000\n(host network, NET_ADMIN)" as API
  storage "webapp-logs volume" as WL
  storage "nginx-logs volume" as NL
  file "/var/log/auth.log" as AUTH
}
actor "Admin / Viewer\nBrowser" as U
actor "Attacker\nBrowser" as ATK
API --> WL : writes login.log
NG --> NL : writes access_json.log
WL --> VEC
NL --> VEC
AUTH --> VEC
VEC --> OS : bulk insert
OSD --> OS
API --> OS : queries / alerts
NG --> API : /api/ proxy
U --> RC : :8090
U --> API : :8000
ATK --> NG : :8080
@enduml
```

Part B – Screenshots (capture from the running system)

Start the system with `docker-compose up -d --build`, wait about a minute, and use a normal browser window at a good size (e.g. 1366 × 768 or larger). Take Figure 7.1 and the dashboard shots **after** running the three attacks, so charts and tables are full. Use Windows *Snipping Tool* / Linux *Screenshot* (Print Screen) and crop to the browser content.

Figure	Screen	Open	Steps	Must be visible
Figure 6.1 Report p.28	Demo Portal – Home Page	http://localhost:8080	Just open the page.	Micro-SIEM hero text, feature cards, 'Enter Demo Portal' button.
Figure 6.2 Report p.29 · Slide 13	Demo Login – Access Denied	http://localhost:8080/login.html	Enter test / wrong and click Authenticate.	Red '✖ ACCESS DENIED – invalid credentials' message under the form.
Figure 6.3 Report p.29 · Slide 13	Search Page with XSS Payload	http://localhost:8080 → log in (admin / 1234) → search	Search for <img src=x onerror=alert(1)> (or <script>alert(1)</script>).	The alert pop-up or the reflected result on search.html.
Figure 6.4 Report p.30	Dashboard Sign-In Page	http://localhost:8090/login.html	Just open the page.	'MICRO-SIEM ACCESS' sign-in card.
Figure 6.5 Report p.30 · Slide 14	Admin Dashboard – Overview	http://localhost:8090 → admin login	First run the 3 attacks from Figure 7.1 so all charts have data.	4 stat tiles, login doughnut, activity line chart, Attack Vectors bar chart with RED brute-force/XSS bars and a BLUE suspicious-login bar.
Figure 6.6 Report p.31	Admin Dashboard – All Logs View	admin.html → 'All Logs' in sidebar	After some activity on the demo portal.	System Event Stream table with nginx_access and web_login rows.
Figure 6.7 Report p.31 · Slide 15	Admin Dashboard – Threats & Action	admin.html → 'Threats & Action'	After the 3 attacks.	Active Alerts table (IP, HIGH, alert type, MITRE ID T1110 / T1078 / CWE-79) and the Blocked panel.

Figure 6.8 Report p.32 · Slide 15	Coming Soon Pop-up	Threats & Action → 'Analyze / Mitigate'	Click 'Execute IP Level Block' (or 'Block User').	The 'Coming Soon' pop-up.
Figure 6.9 Report p.32	Viewer Dashboard (Read-Only)	http://localhost:8090 → viewer login	Log in with the viewer account.	Same stats and charts, 'ROLE: READ-ONLY VIEWER' in the sidebar, no action buttons.
Figure 6.10 Report p.33	OpenSearch Dashboards – Raw Events	http://localhost:5601 → Discover	First create index pattern logs-index* (Dashboards Management → Index patterns).	List of ingested events with fields.
Figure 6.11 Report p.33	Running Containers	Terminal	Run: docker-compose ps	All six containers Up: opensearch, opensearch-dashboards, vector, fastapi-app, nginx-loginpage, react-console.
Figure 6.12 Report p.34	FastAPI API Documentation	http://localhost:8000/docs	Just open the page.	Endpoint list: /login, /session-check, /auth/login, /auth/logout, /logs, /alerts, /block, /unblock, /block-account, /unblock-account, /blocked.
Figure 7.1 Report p.37	Alerts Raised During Attack Simulation	admin.html → Threats & Action (or Dashboard)	From a phone on the same Wi-Fi: 6 wrong logins; then 2 wrong + 1 correct; then an XSS search.	All three alert types with their MITRE / CWE IDs.

Attack simulation for the screenshots (from a phone or second PC on the same Wi-Fi)

1. **Brute force:** open the demo login page and enter a wrong password 6 times within 2 minutes → 'Web Login Brute Force' (T1110).
2. **Suspicious login:** enter 2 wrong passwords, then the correct one (e.g. admin / 1234) → 'Suspicious Login: Failed Attempts Then Success' (T1078).
3. **XSS:** on the search page search for <script>alert(1)</script> → 'XSS Attempt Detected' (CWE-79).

Using a second device makes its real IP appear in the alerts; attacks from the same PC show the Docker gateway IP (172.x.x.1).